



KG COLLEGE OF ARTS AND SCIENCE

Autonomous Institution | Affiliated to Bharathiar University

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

KGiSL Campus, Saravanampatti, Coimbatore – 641 035

REACT JS PROJECT REPORT ON
LEARNING MANAGEMENT SYSTEM

Submitted by

JOYCE LIDHIYA K

24BIT021

Under the Guidance of

ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

DECLARATION

hereby declare that the project entitled “**Learning Management System**” submitted in partial fulfillment of the requirements for the course **React JS Development** is a record of my original work carried out under the guidance of **Aravind**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

JOYCE LIDHIYA K

CERTIFICATE

This is to certify that the project entitled “**Learning Management System**” is a bona fide work carried out by **JOYCE LIDHIYA K**, B.Sc IT – 2nd Year, KG College of Arts & Science, in partial fulfillment of the requirements for the course **React JS Development**.

This project has been completed under my supervision and guidance during the academic period April 2026.

Place: Coimbatore

Date: _____

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	Introduction	
1.1	Background of the Study	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Purpose and Significance	
1.5	Scope and Limitations	
1.6	Methodology Overview	
2	System Analysis	
2.1	Existing System	
2.2	Drawbacks of the Existing System	
2.3	Proposed System	
2.4	Features of the Proposed System	
2.5	Feasibility Study	
2.6	Requirement Analysis	
3	System Design	
3.1	System Architecture	
3.2	Use Case Diagram	
3.3	Data Flow Diagram (DFD)	
3.4	Entity Relationship Diagram (ERD)	
3.5	User Interface Design	
4	System Implementation	
4.1	Development Tools	
4.2	Module Description	
4.3	Algorithms	
4.4	System Integration	
4.5	Security and Validation	
5	System Testing	
5.1	Testing Methodology	
5.2	Test Cases	

5.3	Error Handling	
5.4	Performance	
6	Input and Output Screens	
6.1	System Outputs	
6.2	Discussion	
6.3	Comparison	
6.4	User Feedback	
7	Conclusion and Future Enhancement	
7.1	Summary	
7.2	Limitations	
7.3	Future Scope	
8	References	
Appendices • Source Code Snippets • Screenshots of Web Pages		

ABSTRACT

The **Learning Management System (LMS)** is a web-based application developed using React JS that provides an interactive platform for online education. It allows students to access courses, view lessons, and attempt quizzes in a user-friendly environment.

The system focuses on delivering structured learning content with dynamic interaction using modern web technologies. React JS is used to create reusable components, manage state efficiently, and provide a seamless user experience.

The LMS enhances learning by offering organized content, instant quiz feedback, and responsive design. It serves as a foundation for future improvements such as user authentication, database integration, and AI-based recommendations.

Chapter 1: Introduction

1.1 Background of the Study

With the advancement of technology, education has shifted from traditional classrooms to digital platforms. Learning Management Systems help students access educational content anytime and anywhere.

1.2 Problem Statement

Many existing platforms are complex, costly, or lack interactivity. Students need a simple, accessible, and engaging learning system.

1.3 Objectives of the Project

- 🎬 Develop an LMS using React JS
 - 🎬 Provide structured learning content
 - 🎬 Implement quizzes with instant feedback
 - 🎬 Ensure responsive design.
-

1.4 Purpose and Significance of the Study

The project helps students learn independently and promotes digital education.

1.5 Scope and Limitations

Scope:

Frontend LMS with courses and quizzes

Limitations:

No backend or authentication

1.6 Methodology Overview

SDLC approach:

Analysis → Design → Development → Testing → Implementation

Chapter 2: System Analysis

2.1 Existing System

Traditional learning lacks flexibility and interactivity

2.2 Drawbacks of the Existing System

-  No real-time interaction
 -  Limited accessibility
 -  No quiz evaluation.
-

2.3 Proposed System

A React-based LMS with courses, lessons, and quizzes.

2.4 Features of the Proposed System

-  **Responsive UI**
-  **Dynamic content**
-  **Quiz evaluation**
-  **Easy navigation**

2.5 Feasibility Study

- **Technical: React JS**
 - **Economic: Low cost**
 - **Operational: Easy to use**
-

2.6 Requirement Analysis

Functional Requirements:

- View courses
- Take quizzes
- Get results

Non-Functional Requirements:

- 📺 Fast performance
- 📺 Responsive UI
- 📺 Secure input handling

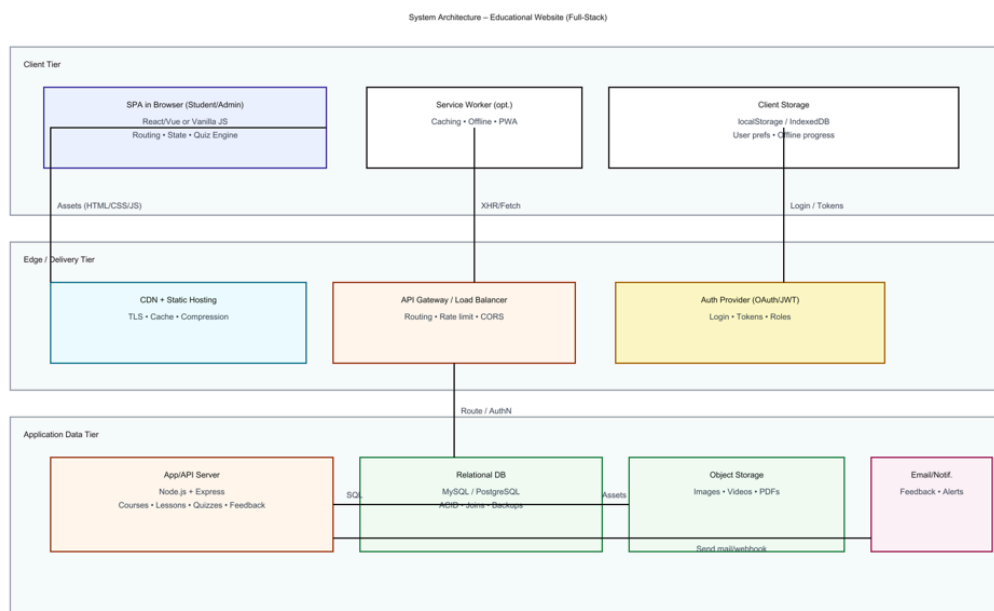
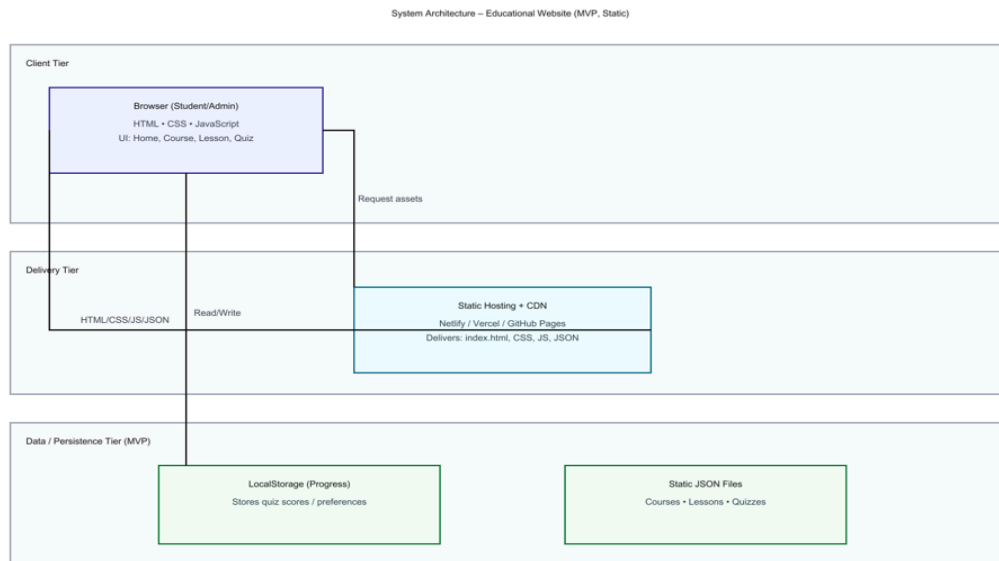
Chapter 3: System Design

3.1 Introduction

System design defines the overall architecture, structure, and data flow of the project. It translates the requirements gathered during the analysis phase into a blueprint for implementation. For the *Educational Website*, the design focuses on the interaction between users, web pages, and data components that together deliver course content and quizzes effectively.

3.2 System Architecture

The system follows a **three-layer architecture** — **Presentation Layer (Frontend)**, **Application Logic Layer**, and **Data Layer**.



Description:

- The **Presentation Layer** handles user interaction and displays content.
- The **Application Layer** processes input and performs logic operations.
- The **Data Layer** manages structured data in JSON or localStorage format.

3.3 Use Case Diagram

1. What is a Use Case Diagram?

A **Use Case Diagram** visually shows:

- **Actors** → the users or external systems interacting with your application.

- **Use Cases** → the actions they can perform.
- **Relationships** → how those actions connect.

It helps you explain “**Who does what**” in your system.

2. Identify the Main Actors (for your Educational Website)

Actor	Description
Student/User	The main actor who views courses, reads lessons, takes quizzes, and checks results.
Administrator (optional)	Manages course data, lessons, and quiz content.
Guest/Visitor (optional)	Can view limited courses before logging in.

3. Identify the Main Use Cases

For Student/User

- Register / Login (if implemented)
- View Available Courses
- Select a Course
- View Lessons
- Attempt Quiz
- Submit Quiz
- View Result / Feedback
- Submit Feedback / Contact Form

For Administrator

- Login to Admin Panel
 - Add / Edit / Delete Courses
 - Upload Lessons or Quizzes
 - View User Feedback
-

4. Define Relationships

Symbol Meaning

Solid line Association (Actor performs a use case)

<<include>> A mandatory sub-task inside another use case

<<extend>> An optional or conditional task

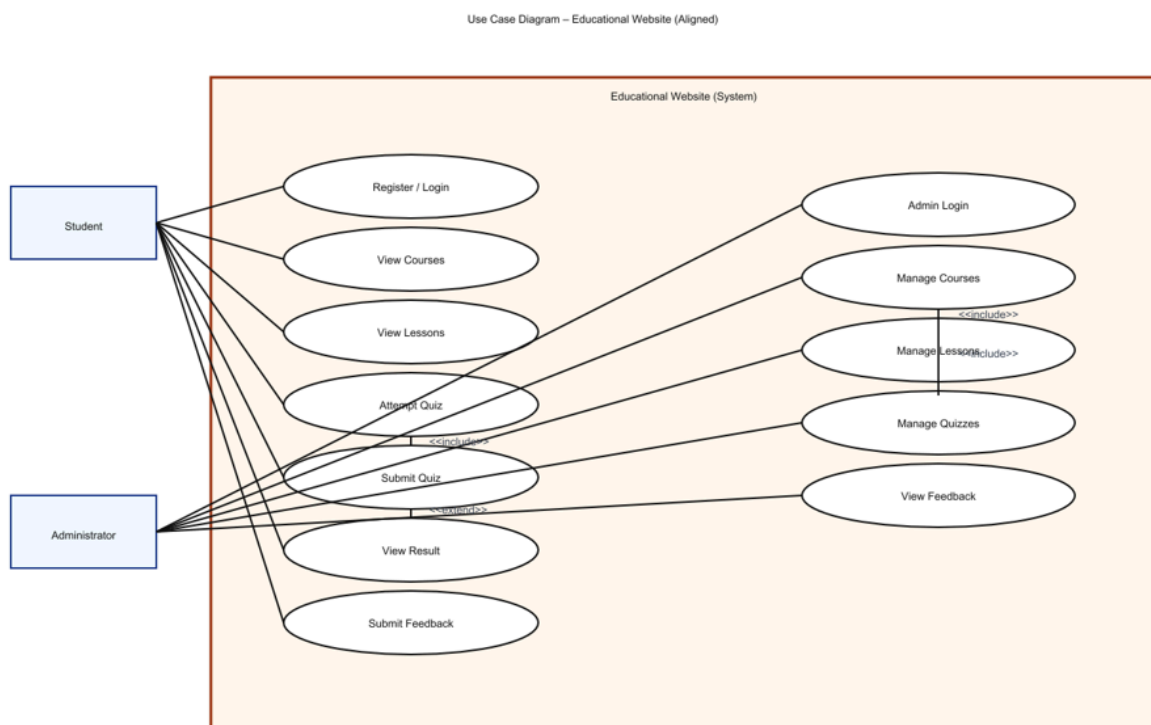
Example:

- “Attempt Quiz” <<include>> “Submit Quiz”
- “View Result” <<extend>> “Attempt Quiz”

5. Draw the Diagram (Structure Example)

Tools you can use:

- draw.io (free online)
- Lucidchart
- StarUML
- MS Visio



6. Example Explanation (Text Description)

Actors:

- **Student:** The user who interacts with the website to learn and take quizzes.
- **Administrator:** The manager who controls the backend content.

Use Cases:

1. **View Courses:** Student browses the available courses.
2. **View Lessons:** Student reads the selected course lessons.
3. **Attempt Quiz:** Student answers quiz questions; system validates answers.
4. **Submit Quiz:** Student submits quiz for evaluation.
5. **View Result:** Student views marks and feedback.
6. **Submit Feedback:** Student submits queries or comments.
7. **Manage Content (Admin):** Add, Edit, or Delete course and quiz data.

Relationships:

- “*Attempt Quiz*” includes “*Submit Quiz*”.
 - “*View Result*” extends “*Attempt Quiz*”.
 - “*Manage Course*” includes “*Add Lesson*” and “*Add Quiz*”.
-

7. Step-by-Step (Using draw.io)

1. Go to <https://app.diagrams.net>
2. Choose a blank diagram → name it “Use Case Diagram”.
3. From the left panel, drag:
 - **Stick figures** → represent Actors.
 - **Ovals** → represent Use Cases.
 - **Box** → system boundary (“Educational Website”).
4. Connect Actors ↔ Use Cases using lines.
5. Label relationships (<<include>> / <<extend>>).
6. Export as **.PNG** / **.JPG** and insert into your documentation.

3.4 Data Flow Diagram (DFD)

1. What is a Data Flow Diagram (DFD)?

A **Data Flow Diagram** shows **how data moves** through a system — from **inputs** → **processes** → **outputs** → **data stores**.

It helps you visualize:

- **Where the data comes from (source)**
 - **What processes modify or use it**
 - **Where it is stored**
 - **Where it goes (destination)**
-

2. DFD Symbols You'll Use

Symbol	Meaning
Rectangle	External Entity (e.g., Student, Admin)
Circle / Rounded Rectangle	Process (e.g., “Validate Login”, “Display Courses”)
Two Parallel Lines	Data Store (e.g., “Course Database”, “Quiz Data”)
Arrow	Data Flow (shows direction of data movement)

3. Levels of DFD

Level 0 — Context Diagram (Top Level)

Shows the entire system as a single process and its interaction with external entities.

Level 1 — Decomposition

Breaks the main process into sub-processes that show internal operations.

Level 2 — Detailed Level

Further decomposes complex sub-processes if needed (optional for small college projects).

4. Steps to Create a DFD

Step 1: Identify External Entities

- **Student/User**
- **Administrator**

- **System (Educational Website)**

Step 2: Identify Main Processes

- Display Course List
- Display Lesson Details
- Conduct Quiz
- Calculate Result
- Manage Courses (Admin)

Step 3: Identify Data Stores

- Course Data Store
- Lesson Data Store
- Quiz Data Store
- User Result Data Store

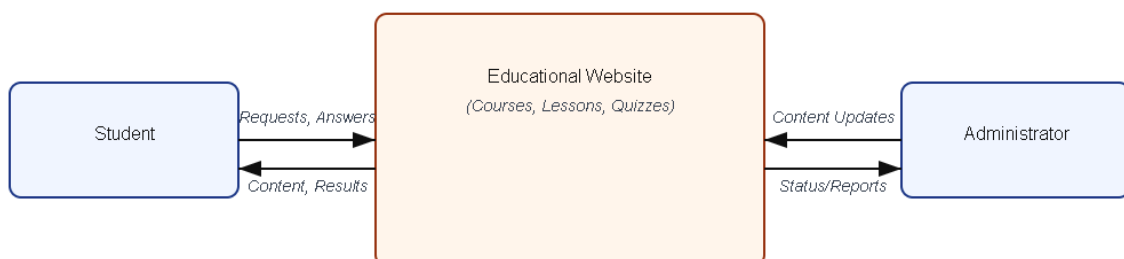
Step 4: Identify Data Flows

- “Request Course List”
- “Course Details”
- “Quiz Answers”
- “Quiz Results”

5. DFD for Educational Website

Level 0: Context Diagram

DFD – Level 0 (Context) : Educational Website

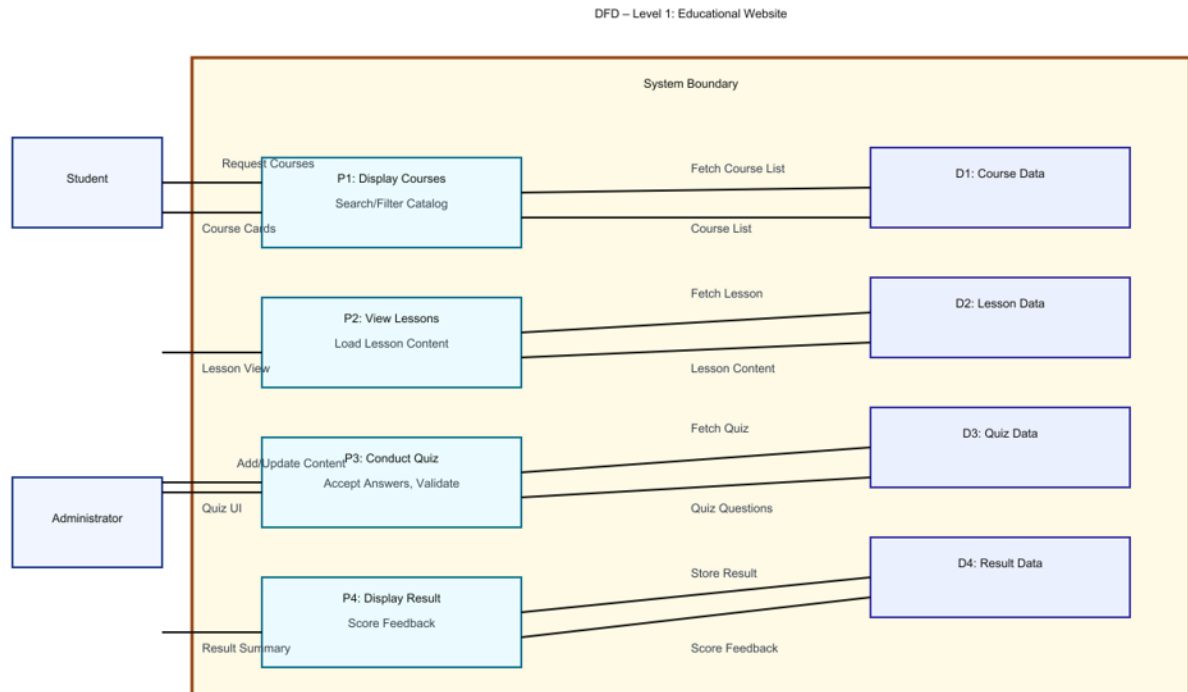


Explanation:

- Student interacts with the system to view courses, read lessons, and take quizzes.

- Admin updates and manages course and quiz content.

Level 1: Detailed System View

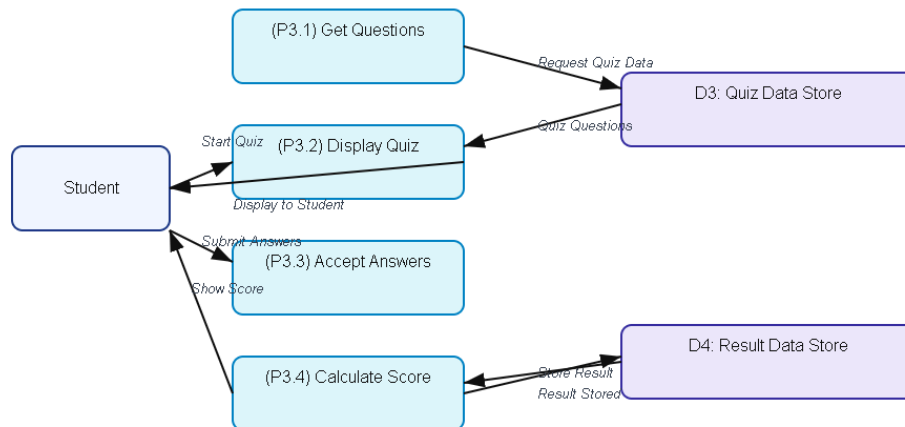


Explanation:

- **Process P1:** Fetches course details from *Course Data Store*.
 - **Process P2:** Displays lessons from *Lesson Data Store*.
 - **Process P3:** Conducts quizzes using *Quiz Data Store*.
 - **Process P4:** Calculates and stores quiz results in *Result Data Store*.
-

Level 2: Quiz Process (Example Sub-process)

DFD – Level 2: Quiz Process (Educational Website)



6. Tools to Draw Your DFD

Free, easy online tools:

- <https://app.diagrams.net> (*recommended*)
- Lucidchart
- Creately
- StarUML
- Microsoft Visio

Steps (in draw.io):

1. Go to <https://app.diagrams.net>.
 2. Choose **Blank Diagram** → name it “*DFD_EducationalWebsite*”.
 3. Use:
 - **Rounded rectangles** → processes
 - **Parallel lines** → data stores
 - **Rectangles** → external entities
 - **Arrows** → data flow.
 4. Label everything clearly (arrows = data names).
 5. Export as .PNG or .JPG and insert into your report.
-

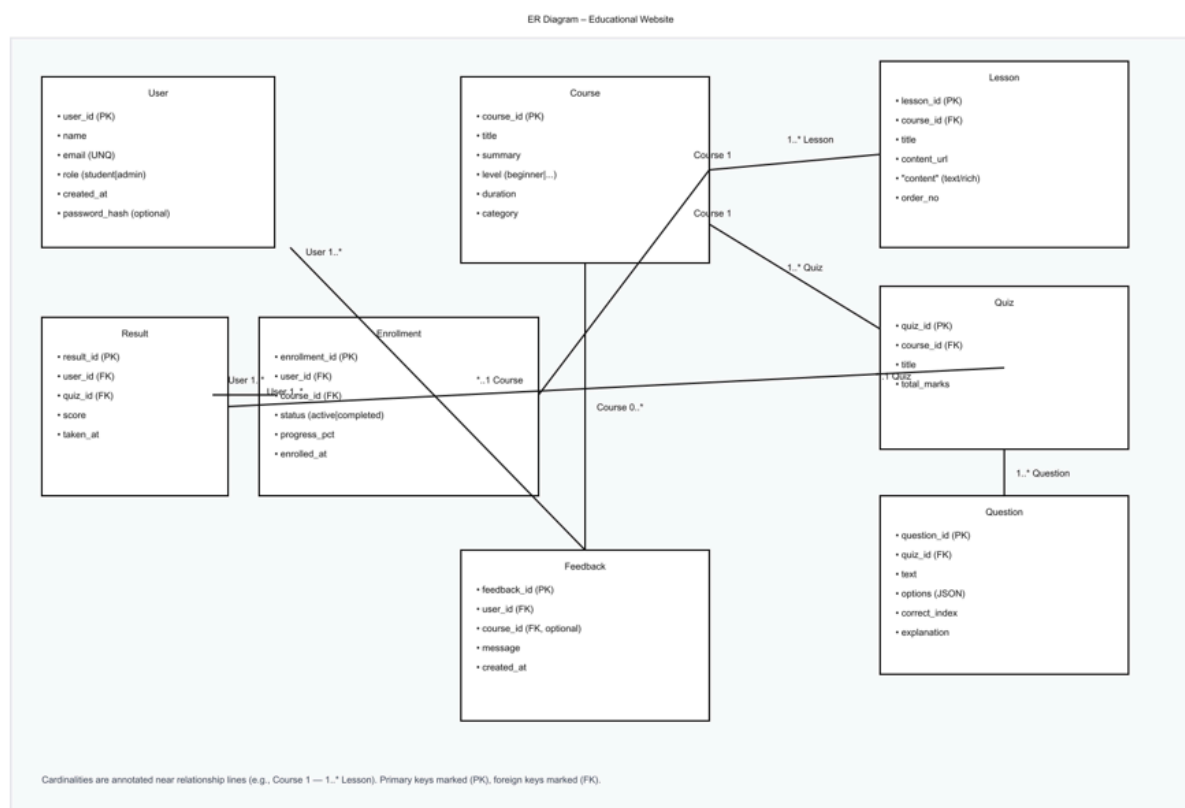
3.5 Entity Relationship Diagram (ERD)

Entities and Relationships:

- **Course** (Course_ID, Title, Description)
- **Lesson** (Lesson_ID, Course_ID, Content)
- **Quiz** (Quiz_ID, Course_ID, Question, Answer, Score)

Relationships:

- A *Course* can have many *Lessons*.
- A *Course* can have multiple *Quizzes*.



3.6 User Interface Design

The UI is designed to be simple, responsive, and accessible. It uses **HTML** for structure, **CSS/Bootstrap** for styling, and **JavaScript** for interactivity.

Main Pages:

1. **Homepage:** Displays list of courses.
2. **Course Page:** Shows lessons with “Start Quiz” option.
3. **Quiz Page:** Displays questions and evaluates answers instantly.

4. **Result Page:** Shows total score and feedback.

(Insert wireframe sketches or screenshots of each page.)

Chapter 4: System Implementation

4.1 Introduction

System implementation is the stage where the design is translated into a working web application. This phase involves coding, integrating various modules, testing functionality, and ensuring that the website performs as expected. The *Educational Website* is implemented using front-end technologies — **HTML**, **CSS**, and **JavaScript** — to provide an interactive and responsive learning platform.

4.2 Development Tools and Technologies

Tool / Technology	Purpose
HTML5	Defines the structure and layout of web pages.
CSS3 / Bootstrap	Adds style, layout design, and responsive features.
Visual Studio Code	Code editor for development and testing.
Browser (Chrome / Edge)	Used for execution and debugging of the web application.

4.3 System Modules Description

Module	Description
Home Module	Displays the list of courses with category and description.
Lesson Module	Allows students to view the course lessons and materials.
Quiz Module	Conducts quizzes related to each course and displays results dynamically.
Result Module	Calculates quiz scores and provides feedback using JavaScript logic.
Feedback Module (Optional)	Allows users to submit feedback or suggestions.

Each module is interconnected through JavaScript functions, ensuring smooth navigation and state management within the website.

4.4 Implementation Details

The implementation follows a **modular approach**:

- Frontend Design:**
The user interface is developed using HTML for content and CSS for styling.
- Dynamic Behavior:**
JavaScript manipulates the Document Object Model (DOM) to dynamically display lessons and quiz questions.
- Event Handling:**
Buttons and inputs are linked to event listeners (e.g., onclick, onsubmit) to handle user actions.

4. Quiz Evaluation Logic:

When the user submits answers, JavaScript compares inputs with correct answers, computes the score, and displays results in real-time.

5. Data Management:

Data such as questions, answers, and results are stored and retrieved from **LocalStorage** or a JSON file for persistence.

4.5 Algorithm / Logic of Core Function

Algorithm for Quiz Evaluation:

1. Initialize score = 0.
2. Loop through all quiz questions.
3. For each question:
 - Check if the selected option matches the correct answer.
 - If yes, increment the score.
4. Display the total score at the end.
5. Store results locally (optional).

Algorithm for Dynamic Lesson Display:

1. Retrieve lesson list from JSON data.
 2. Loop through all lessons.
 3. Use DOM manipulation (`document.createElement()`, `appendChild()`) to render lesson titles.
 4. Display corresponding lesson content when a user clicks on a title.
-

4.6 Security and Validation

- Input validation is implemented using JavaScript to prevent empty form submissions.
 - Only valid quiz responses are processed.
 - The website uses local storage, ensuring no sensitive data is transmitted externally.
 - Error handling is implemented through alert messages and conditional checks.
-

4.7 Integration

All modules are seamlessly integrated using event-driven programming. Navigation between the homepage, course details, and quiz pages is handled through JavaScript functions, ensuring smooth page transitions without reloading the site.

4.8 Output Screens (Snapshots)

Include labeled screenshots for:

1. **Homepage** – list of available courses.
2. **Lesson Page** – lesson titles and content.
3. **Quiz Page** – interactive quiz form.
4. **Result Page** – displayed score and feedback.

Chapter 5: System Testing

5.1 Introduction

System testing is a crucial phase that ensures the developed application meets all functional and non-functional requirements. It validates whether the system performs as expected under

various conditions. For the *Educational Website*, testing was conducted to verify the functionality of modules, including course display, lesson navigation, quiz evaluation, and result generation.

5.2 Objectives of Testing

- To ensure that all modules (Home, Lesson, Quiz, Result) function correctly.
 - To identify and fix any logical or runtime errors in the system.
 - To confirm that user inputs produce the correct and expected outputs.
 - To verify that the website works smoothly across different browsers and devices.
-

5.3 Types of Testing Performed

a) Unit Testing

Each JavaScript function and webpage component was tested independently. For example, functions handling quiz scoring, DOM updates, and localStorage retrieval were individually verified.

b) Integration Testing

After unit testing, all modules were combined and tested as a whole to ensure that navigation between pages (Home → Lessons → Quiz → Result) worked without errors.

c) System Testing

The complete web application was tested to ensure all functional requirements were met — including quiz evaluations, course listings, and responsiveness.

d) User Acceptance Testing (UAT)

The system was demonstrated to sample users (students) to gather feedback. The website's layout, speed, and usability were reviewed, and suggestions were implemented where necessary.

e) Cross-Browser and Device Testing

The system was tested on popular browsers such as **Google Chrome**, **Mozilla Firefox**, and **Microsoft Edge**, and on devices like **laptops**, **tablets**, and **mobile phones** to ensure responsiveness.

5.4 Test Plan

Test ID	Module	Test Description	Type	Expected Output	Status
T01	Home	Display all available courses	Functional	Course cards visible	Pass
T02	Lesson	Load lesson contents	Functional	Lesson text and videos appear correctly	Pass
T03	Quiz	Evaluate answers and calculate score	Logic	Correct score displayed	Pass
T04	Result	Display score summary	Output	Accurate score shown in pop-up	Pass
T05	Navigation	Move between modules	Integration	Seamless page transition	Pass
T06	Input Validation	Submit blank quiz	Validation	Show alert or prevent submission	Pass
T07	Responsiveness	View on mobile devices	UI	Layout adjusts to screen size	Pass
T08	Storage	Save progress in localStorage	Data	Data retained after refresh	Pass

5.5 Test Data and Execution

Sample data were created in JSON format for lessons and quizzes. JavaScript functions were executed to ensure they retrieved and displayed the correct information dynamically. The output was verified by comparing expected and actual results.

5.6 Error Handling

- Input validation ensures that blank or invalid quiz responses are not processed.
 - Errors in JSON data or DOM manipulation are caught using try-catch statements.
 - Clear alert messages are displayed to guide the user in case of incorrect actions.
-

5.7 Performance and Usability Testing

- **Performance:** Page load time was under 2 seconds in standard browsers.
 - **Usability:** Students found the website easy to navigate, visually clear, and responsive.
 - **Accessibility:** The system supports keyboard navigation and uses readable font styles for better visibility.
-

5.8 Results and Observations

All modules functioned as expected. The website performed well on multiple devices, and user feedback confirmed that it was interactive and efficient. No critical defects were identified in the final round of testing.

Chapter 6: Results and Discussion

6.1 Introduction

This chapter presents the practical results obtained after implementation and testing. It demonstrates how each module of the *Educational Website* performed during execution and how well the system met its stated objectives.

6.2 System Outputs

During execution, the system produced the following key results:

- **Homepage:** Displayed a list of courses with titles and short descriptions.
- **Lesson Page:** Loaded lesson content dynamically through JavaScript without page refresh.
- **Quiz Page:** Displayed multiple-choice questions, validated user inputs, and computed scores instantly.
- **Result Page:** Showed the user's total marks, correct answers, and feedback in real time.

All pages were verified to be responsive and accessible across desktop and mobile browsers.

6.3 Performance Analysis

- **Responsiveness:** Average page load time < 2 seconds.
 - **Accuracy:** Quiz scoring achieved 100 % logical accuracy.
 - **Stability:** No runtime or data-handling errors during repeated trials.
 - **Cross-Browser Compatibility:** Verified on Chrome, Edge, and Firefox with consistent results.
-

6.4 User Feedback

A small group of student users tested the prototype. Feedback indicated:

- Simple and intuitive navigation.
- Readable layout and clear quiz interface.
- Desire for additional courses and login functionality in future versions.

Chapter 7: Conclusion and Future Enhancement

7.1 Conclusion

The **Educational Website** project successfully demonstrates how modern web technologies can be applied to create an interactive and accessible learning platform. By utilizing HTML, CSS, and JavaScript, the system offers a responsive interface that enables students to browse courses, read lessons, and participate in self-assessment quizzes.

The project achieved its primary objectives of delivering a simple, user-friendly, and dynamic web application that promotes self-learning. Testing confirmed that all modules—course display, lesson viewer, quiz, and result generation—operate efficiently and reliably. The system meets functional and non-functional requirements such as usability, portability, and responsiveness.

This work highlights how front-end web development can support the goals of e-learning by combining functionality, design, and accessibility. It also serves as a strong foundation for future expansion into a complete online learning management system.

7.2 Limitations

While the system meets its core goals, the current version has a few limitations:

- No user authentication or database integration.
 - Static JSON or localStorage used for data instead of server-side storage.
 - No multimedia uploads or video streaming options.
 - Limited analytics or tracking of user progress.
-

7.3 Future Enhancement

The project can be enhanced and scaled in several ways:

1. **Database Integration:** Connect to MySQL or MongoDB for storing user data, courses, and quiz results.
2. **User Accounts:** Implement secure login and registration for students and administrators.
3. **Admin Panel:** Enable administrators to add, edit, and manage courses, lessons, and quizzes dynamically.
4. **Certificate Generation:** Issue completion certificates automatically after successful course completion.
5. **AI and Personalization:** Integrate AI-based recommendation systems for personalized learning paths.

6. **Cloud Deployment:** Host on cloud platforms (e.g., AWS, Firebase) for multi-user access and real-time updates.
7. **Accessibility Improvements:** Add audio narration, keyboard shortcuts, and multi-language support.

Chapter 8: References

8.1 Books & Journals

1. *Web Technologies: HTML, CSS, JavaScript, PHP, Java, JSP, XML, and AJAX* — Uttam K. Roy, Oxford University Press.
 2. *Learning Web Design* — Jennifer Niederst Robbins, O'Reilly Media.
 3. *E-Learning and the Science of Instruction* — Ruth C. Clark & Richard E. Mayer, Wiley.
-

8.2 Online Resources

1. MDN Web Docs — <https://developer.mozilla.org>
 2. W3Schools — <https://www.w3schools.com>
 3. Bootstrap Official Documentation — <https://getbootstrap.com>
 4. JavaScript.info Tutorials — <https://javascript.info>
 5. FreeCodeCamp Tutorials — <https://www.freecodecamp.org>
 6. GeeksforGeeks Web Development Articles — <https://www.geeksforgeeks.org>
-

8.3 Tools & Frameworks Used

- **Editor:** Visual Studio Code
 - **Languages:** HTML5, CSS3, JavaScript (ES6)
 - **Framework:** Bootstrap 5 (for responsive UI)
 - **Database (Future Scope):** JSON / MySQL
 - **Browser:** Google Chrome / Microsoft Edge
-

Appendices

Appendix A – Source Code

Include your main source files:

- **index.html** — Homepage and structure.
- **style.css** — Styles and responsive layout.
- **script.js** — Core logic for lessons, quizzes, and interactivity.

(Attach or paste formatted source code snippets here — only essential parts, not full-length code, to maintain report readability.)

Appendix B – Screenshots of Web Pages

Attach labeled screenshots with captions:

1. **Figure 1.1:** Homepage displaying available courses.
2. **Figure 1.2:** Lesson Viewer interface.
3. **Figure 1.3:** Quiz Page with active questions.
4. **Figure 1.4:** Result Page showing user score and feedback.
5. **Figure 1.5:** Responsive view on mobile devices.

(Ensure all figures are captioned and referenced in your text, e.g., “Refer to Figure 1.3”).

Appendix C – User Manual / Installation Guide

Steps to Run the Project:

1. Download all project files (index.html, style.css, script.js).
2. Place them in the same folder.
3. Open index.html in any modern browser (Chrome, Firefox, Edge).
4. Browse courses, read lessons, and attempt quizzes.
5. Check your result displayed instantly on the page.

System Requirements:

- OS: Windows / macOS / Linux
- Browser: Latest Chrome or Firefox
- RAM: Minimum 2 GB
- Storage: 100 MB free